Loan Default Risk Assessment: A Development and Deployment Case Study for a Production Machine Learning Credit-Scoring System

James Koero

Independent Machine Learning Engineer, Kisumu, Kenya

Correspondence: jmskoero@gmail.com | github.com/jameskoero/loan-risk-assessment

September 2026

ABSTRACT

This report documents the end-to-end development of a loan default risk assessment system, from an incomplete and undeployed repository to a live, production machine learning service. A gradient boosting classifier was trained on the German Credit Data set (n = 1000) with Synthetic Minority Over-sampling Technique (SMOTE) applied to correct a 70:30 class imbalance, and a decision threshold was selected via a business cost matrix rather than the conventional 0.5 cut-off. The resulting model achieved a five-fold cross-validated receiver operating characteristic area under the curve (ROC-AUC) of 0.916 and a held-out test ROC-AUC of 0.791 (Gini = 0.582), exceeding the Basel III regulatory minimum of 0.35. The report further documents, in full, the deployment failures encountered en route to a working system — a version-controlled artifact exclusion defect, a missing deployment blueprint, a package version mismatch causing a runtime crash, and a prolonged, ultimately network-level rendering fault — together with the diagnostic method used to resolve each. The completed system is served through a REST API and a static web dashboard, both independently verified as live and operational at the time of writing.

Keywords: credit risk modelling; class imbalance; SMOTE; gradient boosting; model deployment; reproducibility; software engineering case study

1. Introduction

Machine learning systems intended for production use are frequently documented at a level of polish that exceeds their actual state of completion. This report presents a case study in which that gap was identified and closed for a single applied system: a loan default risk classifier built on the German Credit Data set. The account given here is deliberately complete rather than selective. Sections 4 through 7 describe not only the final, correct configuration of the system but the sequence of incorrect diagnoses that preceded it, on the view that the diagnostic process is itself part of the engineering record and has instructive value independent of the artifact it produced.

The remainder of this report is organised as follows. Section 2 describes the state of the repository at the outset of the engagement. Section 3 describes the correction of the training pipeline. Sections 4 and 5 describe backend and frontend deployment respectively. Section 6 presents an extended case study of

a rendering fault whose root cause lay outside the repository entirely. Section 7 describes the final system architecture, and Sections 8 and 9 summarise the lessons drawn from the engagement and the closing status of the system.

2. Initial Repository State

At the outset, the repository presented as materially more complete than it was. The README reported an accuracy of 87%, a ROC-AUC of 0.92, and a Gini coefficient of 0.74. These figures corresponded to no run, log, or stored artifact anywhere in the repository's history; they were template defaults, retained from the project's scaffolding and never replaced with measured values. An audit of the repository identified five specific discrepancies between documented and actual state, summarised in Table 1.

Table 1. Discrepancies identified between documented and actual repository state.

Finding	Description
Model artifact exclusion	models/ directory excluded outright in .gitignore; no trained model file had ever been committed.
Missing deployment blueprint	render.yaml, referenced throughout the README, did not exist in the repository.
Missing production server	requirements.txt omitted gunicorn; only the Flask development server could have been run.
Undocumented performance figures	README performance badges had no corresponding run, log, or artifact.
API schema mismatch	Documented request schema (Lending Club style) did not match the fields the training script actually used (German Credit Data).

3. Reconstruction of the Training Pipeline

3.1 Initial retraining run

The training pipeline was first re-executed against the OpenML German Credit Data set in a Google Colaboratory environment. This run produced a cross-validated ROC-AUC of 0.787, a test F1 score of 0.551, and a cost-optimised threshold of 0.11. These results were internally consistent and reproducible, but substantially below the level implied by the repository's own prior documentation. Rather than adjusting the reported figures to align with expectation, the discrepancy was treated as evidence of a missing pipeline step.

3.2 Identification of the missing class-balancing step

The German Credit Data set exhibits a 70:30 imbalance between the non-default and default classes. Inspection of an earlier notebook-based version of the pipeline showed that SMOTE had originally been applied to the training partition prior to model fitting; the script-based version used for the retraining run omitted this step entirely, training directly on the imbalanced data. Re-introducing SMOTE, applied to the training fold only and never to the held-out test partition, resolved the discrepancy: the corrected run achieved a cross-validated ROC-AUC of 0.916 and a test ROC-AUC of 0.791, consistent with the previously undocumented claims.

3.3 Threshold selection under an asymmetric cost matrix

At the model's cost-optimised threshold of 0.22, precision on the default class is 0.45 and recall is 0.833. This combination follows directly from a cost matrix in which a false negative (an approved applicant who defaults) is weighted five times more heavily than a false positive (a rejected applicant who would not have defaulted). This is a deliberate business trade-off, consistent with the risk posture of a conservative lender, and is reported here together with the cost matrix that produced it rather than as an isolated precision figure that might otherwise read as a modelling deficiency.

Table 2. Final verified model performance metrics.

Metric	Value	Basis
Cross-validated ROC-AUC (5-fold)	0.916	SMOTE-balanced training folds
Held-out test ROC-AUC (20%)	0.791	Unseen test partition
Gini coefficient	0.582	$2 \times \text{AUC} - 1$
Optimal decision threshold	0.22	Cost matrix: FN weight 5, FP weight 1
Precision at threshold	0.45	Default class
Recall at threshold	0.833	Default class
F1 at threshold (deployed)	0.585	Default class

4. Backend Deployment

4.1 Version-control artifact exclusion

The first deployment attempt failed at the source-control stage: the trained model artifact could not be committed because .gitignore contained the unqualified pattern `models/`, which excludes the directory in its entirety and prevents git from evaluating any exception rule beneath it. A prior remediation attempt, made through an automated coding assistant, had not corrected this. The effective fix required a different pattern — excluding directory contents rather than the directory itself, so that explicit negation rules could re-include the two required files:

```
models/*
!models/loan_risk_model.joblib
!models/model_metadata.json
```

4.2 Missing deployment configuration

render.yaml, the deployment blueprint referenced throughout the project's documentation, was not present in the repository at any point in its commit history. A blueprint was authored specifying a Python web service, with gunicorn added to requirements.txt as the production WSGI server; the Flask development server referenced in app.py is not suitable for production traffic and was not previously paired with any production server dependency.

4.3 Runtime failure from environment version mismatch

The first live deployment reached the build stage successfully but terminated at runtime with exit status 1. The build log recorded a scikit-learn `InconsistentVersionWarning` immediately preceding the crash. Investigation established that requirements.txt specified the most recent available package versions (scikit-learn 1.8.0, numpy 2.4.4), while the model had been trained and serialised under scikit-learn 1.6.1 and numpy 2.1.3 in the Colaboratory environment. A serialised estimator is not guaranteed to unpickle correctly across a major version boundary, and in this instance did not. The remedy was to query the exact package versions present in the training environment and pin requirements.txt to those precise versions, rather than to the newest available or a loosely compatible range. Following this correction, the service deployed without error and the health-check endpoint returned the expected status.

5. Frontend Development and Deployment

No client-facing interface existed in the repository prior to this work. A dependency-free static dashboard was authored, comprising a single HTML document with a complete applicant intake form covering all twenty fields of the German Credit Data schema, calling the deployed API directly via client-side JavaScript.

The initial deployment to the hosting platform returned an HTTP 404 response in place of the dashboard. Diagnosis established that the project's root-directory configuration was correct, but a residual configuration file, authored under an assumption about repository layout that no longer held, conflicted with that setting. Removing the unnecessary configuration file resolved the fault; a single static document requires no build configuration once the root directory setting is correct. A subsequent end-to-end test, comprising form submission from the deployed dashboard, a live request to the deployed API, and receipt of a computed risk score, confirmed the system operative in full.

6. Documentation Integrity

With the system independently verified as operational, documentation was revised under the principle that every stated claim should be traceable to a specific artifact or run. Five corrections followed from applying this principle systematically, listed in Table 3.

Table 3. Documentation corrections applied following system verification.

Item	Correction applied
Performance badges	Replaced template figures (87% / 0.92 / Gini 0.74) with measured values from the SMOTE-corrected run.
API reference	Rewritten to match the German Credit Data schema actually accepted by the deployed /predict endpoint.
Cross-referenced metric	A related project's README conflated cross-validation-fold accuracy with hold-out accuracy; the two were separated and correctly labelled.
Version-control hygiene	Twenty-two stale branches, chiefly automated pull-request branches, were identified and removed.
Repository metadata	Homepage URL and short description, both pointing to placeholder values, were corrected via the platform API.

7. Extended Case Study: A Persistent Rendering Fault

This section is reported in greater detail than the preceding ones. It concerns the longest and least tractable fault encountered during the engagement, and its resolution illustrates a general diagnostic principle applicable beyond this project.

7.1 Symptom

Following the metrics correction described in Section 3, five evaluation charts embedded in the README — a confusion matrix, a receiver operating characteristic curve, a feature importance plot, and two SHapley Additive exPlanations (SHAP) plots — began intermittently failing to render on the repository's mobile web interface. The specific subset of images affected varied unpredictably between successive page loads.

7.2 Hypotheses evaluated and rejected

Four candidate explanations were formulated and tested in sequence, each disconfirmed by direct evidence before the next was considered:

- *File corruption.* Rejected: every affected file opened and passed integrity verification, with correct dimensions and valid image headers.
- *Content-addressed cache persistence tied to filename.* Renaming the affected files altered which images failed without resolving the underlying fault.
- *Proxy timeout attributable to file size.* Rejected by direct measurement: timed HTTP retrieval of all five files returned in approximately 0.15 seconds uniformly, independent of file size.
- *Malformed markdown syntax.* Rejected by byte-level inspection of the source file; all five image reference lines were structurally identical.

7.3 Root cause

The fault was ultimately localised by requesting a raw image URL directly, bypassing the rendered document entirely. This request returned an explicit intermediary error: `Error 503 Backend.max_conn reached`, issued by a regional Varnish cache server operated by the developer's mobile network provider. The fault lay in a transparent caching proxy on the client's network path reaching its own connection ceiling, entirely independent of the repository, the hosting platform, and the source content. Requests originating from a network path that did not traverse this proxy succeeded uniformly.

7.4 Discussion

No further repository-side modification was required once the network-layer cause was established. One genuine, independent defect — an unclosed code-fence delimiter, a residue of an earlier failed edit — was identified and corrected in the course of this investigation and is retained as a legitimate fix. The broader methodological finding is that an intermittent fault uncorrelated with any change to the artifact under test is better diagnosed by direct inspection of the transport layer than by iterative revision of the artifact; several plausible repository-side changes were made and deployed before this test was

performed, none of which addressed the actual cause.

8. Final System Architecture

Table 4 summarises the components of the deployed system as verified at the time of writing.

Table 4. Deployed system architecture by layer.

Layer	Technology	Function
Model	scikit-learn 1.6.1, GradientBoostingClassifier	Core prediction algorithm
Class balancing	imbalanced-learn 0.14.2 (SMOTE)	Corrects 70:30 class imbalance
API	Flask 3.1.3 with gunicorn 23.0.0	Serves /health and /predict endpoints
Backend hosting	Render (free tier)	Executes the live API
Frontend	Static HTML/JavaScript, no build step	Applicant intake interface
Frontend hosting	Vercel	Serves the dashboard
Training environment	Google Colaboratory	Model training and evaluation
Version control	Git / GitHub	Canonical source of code and artifacts

Both public endpoints were independently confirmed live at the time of writing: the API at `loan-risk-assessment-nsdw.onrender.com` (health check returning the expected status) and the dashboard at `jameskoero-loan-risk-assessment-h84.vercel.app` (confirmed by live end-to-end submission returning a computed risk score).

9. Lessons Learned

- **Documentation constitutes a claim, not a description.** A stated performance metric is an assertion that should be traceable to a specific run; where it cannot be traced, it should not be published.
- **A trained model and its serving environment form a single system.** Version drift between training and serving environments is a common and fully preventable class of production failure.
- **Directory-level exclusion patterns silently negate exception rules beneath them.** Excluding contents rather than the directory itself is the pattern that permits selective re-inclusion.
- **An intermittent, artifact-independent fault warrants direct transport-layer testing before further artifact revision.** A single raw request would have identified the network-layer cause immediately, in advance of several rounds of ultimately unnecessary repository changes.
- **A cost-optimised decision threshold alters precision and recall by design, not by defect.** Reporting such a figure together with the cost matrix that produced it is necessary for correct interpretation.

10. Conclusion

The system described in this report is complete and independently verified: a trained classifier with reproducible, measured performance; a live prediction API; a live user-facing dashboard; and documentation in which every quantitative claim is traceable to an artifact. The path to this state was not linear, and this report has aimed to record that path as it actually occurred, including the diagnostic errors made along the way, rather than a retrospectively smoothed account of it.